Ahead of the Spike — Codebase Guide

Oct 25, 2026 · @omos

1. The project in one page

We forecast which Wikipedia pages people will read in the coming hours and use that forecast to decide what a fixed-size cache should hold. On 72 hours the models never saw, our XGBoost model beats the standard caching rule, and its advantage grows the further ahead the decision has to be made.

The problem. Every Wikipedia page view is served either from a fast cache or from the slower main servers. A cache has limited room, so someone has to choose which pages to keep in it. The standard choice is "keep what was popular most recently". It is simple and surprisingly good, because attention is mostly stable from one hour to the next.

The question. If a cache can hold only the top N pages, can predicting tomorrow's attention beat "keep what was popular last hour"? And by how much, compared with the best anyone could possibly do?

The answer (latest run, from `key_numbers.md`: 72 test hours the models never saw):

- With room for 10,000 pages, deciding 1 hour ahead: XGBoost serves 40.59% of views from cache, the standard rule 40.47%, and perfect foresight 41.66%. The standard rule leaves only 1.19 points on the table. XGBoost's +0.12 is small but consistent: it beat the standard rule in 94% of test hours.
- Deciding 6 hours ahead, XGBoost is **+1.28 points** better (95% interval +0.93 to +1.49) and better in every single test hour. That is about 30% of everything it was possible to gain.
- With room for 100,000 pages, XGBoost recovers 30% of the gap even at 1 hour ahead.
- After a spike, extra attention fades as a power law with a **half-life of about 25 minutes**, measured on 1,429 spikes, and partly returns **23 hours later**.
- The surprise: the gain does **not** come from catching spikes. The standard rule already serves 93.5% of spike traffic, slightly more than XGBoost (92.8%). The gain comes from ranking ordinary pages better, and it grows as the last hour goes stale.

How it fits the HRT brief ("use data to predict and make optimal decisions"):

1. **Predict:** forecast each page's views 1, 2, 3 and 6 hours ahead, with a physics-style decay model and a machine-learning model that uses the physics as an input.

2. **Decide:** fill the cache with the pages predicted to be most viewed.
3. **Measure the decision, not just the forecast:** replay real history and count how many real views each choice would have served, against the standard rule and against perfect foresight.

How it fits the time theme. Time appears three ways: how far ahead the cache must be decided (the horizon), how fast attention decays after a spike, and the daily rhythm of attention. The headline finding is about time itself: the earlier you must commit, the more prediction is worth.

Why judges can trust it. Real Wikipedia data; models trained only on hours before the test period; the test compares methods on exactly the same hours; a perfect-foresight ceiling shows how much was possible; and every claim comes with a confidence interval.

2. Core concepts you must be able to explain

Every number in the pitch is a hit rate measured on a replay of real hours, so these few ideas carry the whole project. Learn them before anything else.

The cache decision

At each hour `now`, a method looks at the past and names the **N pages** it expects to be busiest at hour `now + h`. Those N pages go in the cache. Then we look at what actually happened at `now + h`.

- **N (cache size):** how many pages fit. We test 1,000, 10,000 and 100,000. A real CDN node is limited by memory, so N stands in for "how much fast storage you have". In our data the top 10,000 pages take about 40% of the hour's views; that long tail is why caching works at all.
- **h (horizon):** how many hours ahead the decision is made. We test 1, 2, 3 and 6. A longer horizon is harder for everyone, but it reflects real constraints: pre-warming caches, pushing content to edge servers, scheduling capacity.

Hit rate

Hit rate = views of pages that were in the cache ÷ all views that hour. "All views" means all views in our data, which is the top 150,000 pages each hour (section 3), not the whole of Wikipedia. It is weighted by views, not counted by pages, because one hit on a page with 50,000 views matters 50,000 times more than a page with one view. We average it over every test hour.

Why hit rate and not forecast error: we care about the decision. A model can have a worse average error yet choose a better cache, because only the ranking near the cut-off (page N vs page N+1) changes the outcome. Measuring the decision directly answers "would this help?".

Oracle (the ceiling)

The oracle cheats: it fills the cache with the pages that *actually were* the top N at `now + h`. No real method can beat it. It tells us how much there was to gain. In caching theory this is the idea behind Belady's optimal algorithm (evict whatever is needed furthest in the future), which also needs perfect foresight.

Gap and share of gap recovered

The standard rule, `last_hour`, is our reference. The **gap** is oracle − last_hour: the most any method could possibly add. At 10k pages, 1 hour ahead, it is only 1.19 points, because last-hour popularity is already a strong predictor.

Share of gap recovered = (model − last_hour) ÷ (oracle − last_hour). 0% means no better than the standard rule; 100% means perfect; negative means worse. It puts different N and h on a common scale, which raw points cannot do (a 0.3-point gain is small when the gap is 4 points and large when it is 1).

Walk-forward evaluation

The backtest replays time. At each test hour it hands a method only the rows with `hour ⩽ now`, asks for its prediction, then scores it against `now + h`. Nothing from the future can enter a prediction. This is the same discipline as backtesting a trading strategy: the strategy can only trade on what was known at the time.

Leakage and purging

Leakage means future information sneaking into training. Two guards:

1. **Held-out test period.** The last 3 days (`--test-days 3`) are never used to fit the decay curve or train XGBoost.
2. **Purging.** A training row at hour t uses the target at t + h. Training stops so that even the target hours fall before the test period. Without this, a 6-hour model trained up to the test start would have seen 6 test hours as labels.

The practical rule: **`fit_decay.py`**, **`train_xgb.py`** and **`backtest.py`** **must all be run with the same `--test-days`.** If they differ, the models may have trained on the hours they are scored on, and the results are invalid.

Spikes

A **spike** is a page that suddenly gets far more attention than usual. Our definition, used everywhere: views ≥ 1,000 **and** ≥ 10× the page's median over the previous 24 hours (missing hours count as 0). Only the first hour (the onset) counts as the spike event. We expected spikes to be where the standard rule fails, since it can only cache a page once it is busy and keeps it after it fades. Measured at 10,000 pages, the standard rule actually handles them well (93.5% of spike traffic served); section 11 explains.

3. The data

We use Wikimedia's public hourly pageview dumps for English Wikipedia, cut down to the 150,000 busiest articles each hour and stored as one table: `hour`, `page`, `views`.

Where it comes from

Wikimedia publishes one compressed file per hour at `https://dumps.wikimedia.org/other/pageviews/YYYY/YYYY-MM/pageviews-YYYYMMDD-HH0000.gz`. Each line is `domain title views bytes`, for example `en Tom_Skerritt 61234 0`. A file covers every Wikimedia project and language, about 400–500 MB unpacked, and appears roughly an hour after the hour ends.

How we clean it (`download.py`)

Step	What	Why
Keep `en` and `en.m`	English desktop + mobile, summed per title	Same article, two front doors; a cache serves both
Drop non-articles	Titles with a namespace prefix (`Special:`, `File:`, `Talk:`…), `Main_Page`, `-`	They are site plumbing, not attention; `Main_Page` alone would always top the cache

Step	What	Why
Keep top 150k per hour	Sort by views, keep the head	Below that, pages get a handful of views; they never affect a cache of ⩽100k but would multiply the data size
Parse views as text, then convert	`pd.to_numeric(errors="coerce")`	A few raw lines are malformed and crashed the first version

The hour stamp is **UTC at the start of the hour**: the file `pageviews-20261022-140000` becomes `hour = 2026-10-22 14:00`. All code uses UTC, so the clocks going back did not matter.

Why the download code is so defensive

The venue wifi kept dropping. `download.py` therefore:

- resumes a half-finished file (HTTP `Range` header plus a `.partial` file), retrying with backoff up to 120 s;
- re-downloads a file that fails to decompress;
- saves each finished hour atomically as `data/hours/YYYYMMDD-HH.parquet` and deletes the raw file, so a crash never leaves a half-written hour;
- with `--skip-existing`, picks up where it left off, and with `--order newest` fetches recent hours first;
- reports "not published yet" for the newest hours instead of treating a 404 as a failure.

Combining downloads from several laptops (`combine_parquets.py`)

Three laptops downloaded different stretches with different scripts. `combine_parquets.py` takes parquet files, folders or zips; renames common column aliases to `hour` / `page` / `views`; and when two sources have the same (hour, page), keeps the **maximum** and reports how many disagree. It re-applies the top-150k cut (one teammate's files had ~277k rows per hour), backs up any existing `views.parquet`, and writes `data/missing_hours.txt` listing gaps as ranges. Hours with far fewer rows than usual are flagged as "thin" (likely a truncated download).

How missing hours are handled

A missing hour is **unknown**, not zero. This matters: if a download gap were read as zero views, every page would look like it crashed and recovered, and the decay model would see thousands of fake spikes.

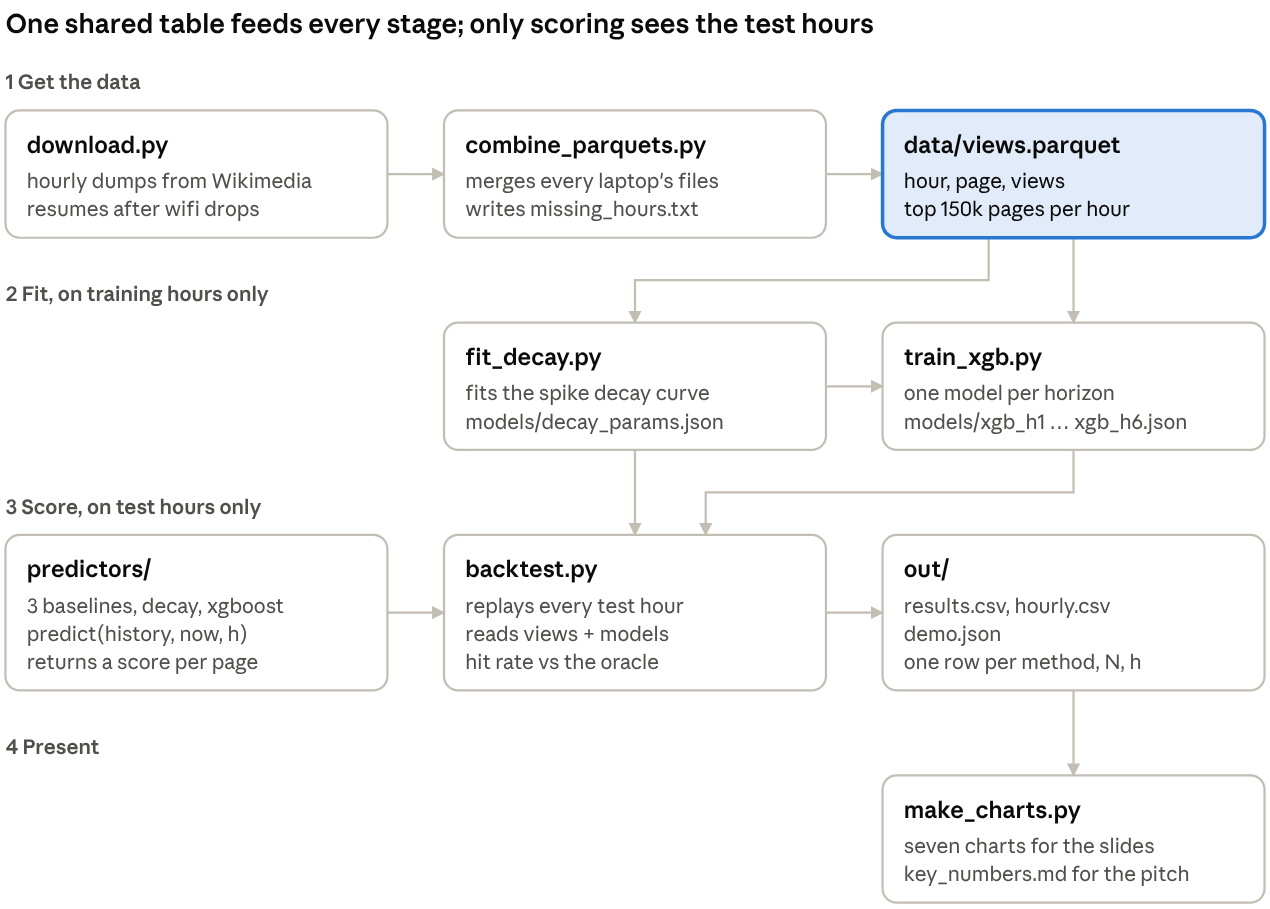
- Baselines use the latest hour that *exists*.
- The decay model builds its 24 h table only from hours that exist.
- XGBoost features put NaN in a missing column; XGBoost learns a default direction for missing values.
- Training skips hours that are missing or that sit in a gap (at least 75% of the 25-hour window must exist).
- The backtest only scores target hours we have.

Limits of the data

- **About a week of data** (the full set runs 16 Oct 09:00 – 24 Oct 23:00 UTC). One weekend at most, so weekly patterns cannot be learned.
- **Top 150k cut.** A page that jumps from 20 views to 2,000 appears "from nowhere": its earlier hours were below the cut, so we record 0. This slightly exaggerates how sudden spikes look, the same way for every method.
- **Hit rate is relative to our data**, the top 150k pages, not all of Wikipedia. Absolute hit rates would be lower with the full tail included; differences between methods would barely move, because the tail is outside every cache.
- **Views are not requests.** The dumps count page views (with automated traffic partly filtered by Wikimedia), not every HTTP request, image or API call a real CDN serves.
- **Hourly resolution.** Real caches decide in milliseconds. We model the hourly "what should be warm" decision, not request-level eviction.

4. How the pieces fit together

The codebase is a straight pipeline around one shared table, `data/views.parquet`, and one shared function signature for predictors. Those two contracts, written down in `INTERFACES.md`, are what let four people build parts in parallel overnight.



pipeline · 4 stages, 9 parts

The decay fit also feeds XGBoost: the decay model's prediction is one of XGBoost's inputs (the `decay_pred` feature, section 9).

File map

File	Owner	What it does
`INTERFACES.md`	all	The contract: data format, predictor signature, backtest rules, demo.json schema
`download.py`	EIE / Computing	Fetches and cleans hourly dumps, one parquet per hour
`combine_parquets.py`	Computing	Merges downloads into `data/views.parquet`, reports gaps

File	Owner	What it does
`make_fake_data.py`	Computing	Realistic fake data so everyone could build before real data arrived
`predictors/baselines.py`	Computing	The three simple rules (section 7)
`predictors/decay_predict.py`	Physics	Spike-decay forecast (section 8)
`fit_decay.py`	Physics / Computing	Fits the decay curve on training spikes
`predictors/xgb_features.py`	Computing	Feature recipe shared by training and prediction
`train_xgb.py`	Computing	Trains one XGBoost model per horizon
`predictors/xgb.py`	Computing	Loads the models, predicts during the backtest
`predictors/__init__.py`	Computing	The registry: which predictors the backtest runs
`backtest.py`	Computing	Walk-forward scoring, spikes, confidence intervals, demo.json
`make_charts.py`	Computing / Design Eng	Slide charts and `key_numbers.md`

The two contracts

The table. One row per (hour, page): `hour` (UTC, start of hour), `page` (title with underscores), `views` (desktop + mobile). A page missing from an hour that exists means it was below the top-150k cut, which we treat as 0. An hour missing entirely means unknown.

The predictor. Every method is a Python function

```
def predict(history: pd.DataFrame, now: pd.Timestamp, horizon: int = 1) ->
pd.Series:
    ...  # index = page, value = score; higher = more likely to be busy at
now + horizon
```

`history` holds only rows with `hour ⩽ now` (the last 8 days by default, `--history-days`). The backtest caches the top N by score. The score does not need to be a calibrated forecast; only its order matters. A predictor whose answer does not depend on the horizon can set `horizon_independent = True`, and the backtest computes it once per hour instead of four times.

Adding a method is two steps: write the function, then add it to `PREDICTORS` in `predictors/__init__.py`. XGBoost registers itself only if its model files exist, so the backtest still runs on a fresh clone.

Commands, in order

```
python combine_parquets.py data/hours other_laptop.zip   # only if data came
from several places
python fit_decay.py   --test-days 3
python train_xgb.py   --test-days 3
python backtest.py    --test-days 3
python make_charts.py
```

The order matters: `fit_decay.py` writes the decay parameters that both the decay predictor and XGBoost's `decay_pred` feature read, so it must run before `train_xgb.py`. If you refit the decay curve, retrain XGBoost. **Use the same `--test-days` everywhere** (section 2).

5. The backtest (`backtest.py`)

The backtest is the referee: it replays the test period hour by hour, asks every method for its cache, and scores each cache against what really happened. If you understand this file you can defend every number in the pitch.

The loop

For every test hour `now`, horizon `h` in {1, 2, 3, 6} and method:

1. Take `history` = rows with `now − 8 days < hour ⩽ now` (found by binary search on the sorted hour column, so it is fast).
2. `pred = method(history, now, h)`. The oracle instead uses the real views at `now + h`.
3. Keep the top `max(N)` pages by `pred`.

4. Look up their real views at `now + h`, in rank order, and take a running total (`np.cumsum`). The hit rate for any N is the running total at position N divided by the hour's total views. One sort serves all three cache sizes.
5. Record where each spike page ranked (for the spike metrics) and, on the replay day, which pages were cached (for the demo).

The **test period** is the last `--test-days` days: from `last − 3 days + 1 h` to `last`. A target hour is scored only if it falls inside the test period and exists in the data. The first `now` is the hour before the test period starts, so the first target is the first test hour.

What it writes

File	Contents	Used for
`out/results.csv`	One row per method × N × h: mean hit rate, 95% interval, median catch lag, spike hit rate, number of hours	Every headline number
`out/hourly.csv`	Hit rate for every method × N × h × target hour	Paired significance tests (section 6)
`out/demo.json`	The replay day hour by hour: top 50 pages, which were cached by `last_hour` and by our model, the day's biggest spikes	The live demo page

The terminal summary prints the hit-rate table at 1 hour, the gap the standard rule leaves, each method's share of it recovered, and the spike metrics.

Spike metrics

Spikes are found with `find_spikes` (definition in section 2) on the test hours, using the 24 hours before the test period for the first medians.

- **Spike hit rate** (the one we use): of all views that spike pages received in the first 6 hours after onset, what share was served from cache? It is weighted by views, like the main hit rate, and focuses on the moments where the standard rule is weakest.
- **Median catch lag** (kept, but not used on slides): hours from onset until the page enters the cache, capped at 24. At N = 10,000 almost every method catches almost every spike within one hour, because a spike of ≥ 1,000 views is usually already in the top 10k.

> The metric therefore cannot tell methods apart, which is why we switched to spike hit rate.

The demo replay

The replay day is the last test day with at least 23 hours of data. For each hour of it, `demo.json` stores the 50 busiest pages and whether each method had them cached at N = 10,000 (decided 1 hour ahead). "Our model" is whichever non-baseline method scored best on that day; the file records its name in `our_model_name`.

Speed choices

The first version took too long on real data. Four changes fixed it without changing any result:

Change	Effect
Keep only the top 150k pages per hour on load (`--top`)	Smaller table, same scores (caches are ⩽ 100k)
Pass only the last 8 days of history (`--history-days 8`)	Every method uses ⩽ 25 hours anyway
Compute horizon-independent methods once per hour	`last_hour` and `last_24h_mean` run 1× instead of 4×
Score all N from one cumulative sum	One sort per method per hour

It also prints progress with an estimated time left, so a slow run is visible rather than looking frozen.

Options you will actually use

Flag	Default	Notes
`--test-days`	3	Must match `fit_decay.py` and `train_xgb.py`
`--sizes`	1000 10000 100000	Use small sizes (e.g. 100 300 1000) on fake data
`--horizons`	1 2 3 6	

Flag	Default	Notes
`--predictors`	all registered	e.g. `--predictors decay` for a quick run; `last_hour` is always added
`--demo-size`	10000	The N used for the demo and the spike summary

Why this design is fair

- **Same hours for everyone.** Every method is scored on exactly the same (now, target) pairs, which is what makes paired comparisons valid.
- **No peeking.** Methods receive only `hour ⩽ now`; the oracle is the only thing that sees the target hour, and it is labelled as a ceiling, not a competitor.
- **The decision is scored, not the forecast.** A method can only win by choosing a cache that serves more real views.

6. Statistics: how we know a difference is real

The differences we claim are a fraction of a percentage point, so we test them on the same hours with a paired block bootstrap. The plain per-method intervals in `results.csv` overlap and would wrongly suggest nothing is different.

Why per-method intervals mislead here

`results.csv` gives each method a 95% interval from an ordinary bootstrap: resample the test hours with replacement 1,000 times, take the mean hit rate each time, keep the middle 95%. Those intervals are wide compared with the differences between methods, because **hours differ a lot from each other**: the time of day changes how many pages share the attention, and a big news hour concentrates views on a few pages. That hour-to-hour swing moves every method up and down together.

Two methods with overlapping intervals can still differ reliably. If XGBoost beats `last_hour` by 0.3 points in nearly every hour, the swing between hours is irrelevant to that comparison. Comparing the two intervals throws away the fact that they were scored on the same hours.

The paired test (`make_charts.py`)

1. From `hourly.csv`, for each test hour, compute **d = method's hit rate − last_hour's hit rate** (in points). The shared hour-to-hour swing cancels out.
2. The estimate is the mean of d over all test hours.
3. Its 95% interval comes from a **moving block bootstrap**: pick random runs of 6 consecutive hours, glue enough of them together to make a series as long as the original, take its mean; repeat 2,000 times; keep the middle 95%.
4. If the interval excludes 0, the difference is labelled significant; if not, the chart writes "n.s." (not significant). We also report the share of hours in which the method beat `last_hour`.

Why blocks of 6 hours

Neighbouring hours are correlated: if XGBoost does well at 14:00 because a news story is unfolding, it probably also does well at 15:00. An ordinary bootstrap treats every hour as independent evidence, so it would overstate how much evidence 72 hours contain and give intervals that are too narrow. Resampling whole 6-hour runs keeps that short-range correlation inside each block. Six is a judgement call: long enough to cover a typical spike's main decay (section 8), short enough to leave about 12 blocks in 72 hours.

How to say it in the pitch

- Good: "On the same 72 hours, deciding 6 hours ahead, XGBoost beat the standard rule by +1.28 points, 95% interval +0.93 to +1.49, and was better in every one of those hours."
- Good: "Even 1 hour ahead, where the most anyone could gain is 1.19 points, XGBoost's +0.12 has an interval of +0.10 to +0.14 and wins 94% of hours. Small, but real."
- Honest: "Our physics model on its own is significantly *worse* than the standard rule (−0.19 points at 1 hour, interval −0.22 to −0.15). It earns its place as an input to XGBoost and as an explanation."
- Avoid: "significantly better" without the interval, or comparing the overlapping bars in `results.csv` / chart 6.

Limits of the statistics

- **72 test hours is three days.** Three days of news is not a representative sample of all days; a quiet weekend or a huge breaking story could change the picture. The interval measures noise within these days, not how typical the days are.

- **Many comparisons.** We look at 3 cache sizes × 4 horizons × several methods. Some "significant" results could appear by chance. Our headline (gain grows with horizon, strongest at 100k) is a consistent pattern across many cells, which is more convincing than any single cell.
- **Block length is a choice.** Longer blocks give somewhat wider intervals. If a judge asks, we can rerun with 12-hour blocks; an effect that only survives with short blocks is weak.

7. The baselines (`predictors/baselines.py`)

The baselines are the simple rules a real cache would use without any forecasting; beating the best of them, `last_hour`, is the bar for every claim we make.

Method	Rule	Classic cache equivalent	Result at 10k, 1 h
`last_hour`	Score = views in the latest hour we have	LRU (least recently used): keep what was just requested	40.47% (the reference)
`last_24h_mean`	Score = total views over the last 24 h ÷ hours we have	LFU (least frequently used): keep what is requested most often	39.12%
`same_hour_yesterday`	Score = views at the target hour minus 24 h	A purely seasonal rule	36.66%

Why these three

Each one stands for a different belief about attention:

- **`last_hour`: attention is persistent.** What people read now they will read next hour. It is the strongest baseline because hourly attention changes slowly for most pages. It is our stand-in for LRU, the default policy in most real caches. (A true LRU works request by request; at hourly resolution "most recently popular" is the closest analogue.)
- **`last_24h_mean`: attention is stable over a day.** Averaging smooths noise but reacts slowly: a page that spiked 20 hours ago is still rated highly, and a page spiking now is diluted by 23 quiet hours. That is why it loses to `last_hour`.

- **`same_hour_yesterday`: attention follows a daily rhythm.** It captures time-of-day patterns but ignores everything that happened in the last 23 hours, including every new story. It does worst.

Together they bracket the problem: the best predictor must be as reactive as `last_hour`, but also know when a spike is fading (which `last_hour` cannot) and when the daily rhythm is about to lift a page (which only `same_hour_yesterday` sees). XGBoost gets all three as features and learns when to trust which.

Implementation details worth knowing

- **Binary search.** `history` is sorted by hour, so `_slice` finds the rows for any hour range with `np.searchsorted` instead of scanning the whole table.
- **Missing hours.** `last_hour` uses the latest hour that *exists*, not `now`, so a download gap never produces an empty cache. `last_24h_mean` divides by the number of hours present, not by 24. `same_hour_yesterday` falls back to `last_hour` if the hour 24 h before the target is missing.
- **Horizon.** `last_hour` and `last_24h_mean` give the same answer for every horizon and are marked `horizon_independent`. `same_hour_yesterday` does use the horizon: 6 hours ahead, it looks at the hour 18 hours ago.

Why `last_hour` is hard to beat

The oracle at 10k, 1 h is 41.66%; `last_hour` already gets 40.47%. The total room for improvement is 1.19 points. This is not a weakness of the project; it is the finding that makes the rest meaningful. Most attention is predictable from the last hour, so the value of forecasting is concentrated in the minority of pages whose attention is changing, and it grows when the decision must be made further ahead (section 11).

But `last_hour` goes stale. At 6 hours ahead the 24-hour average *beats* `last_hour` by +0.53 points (chart 1), while at 1 hour it lost by 1.35. The single latest hour is the best summary of the next hour, but a smoother summary is better for six hours from now. `same_hour_yesterday` also closes most of its gap (−3.81 at 1 hour, −0.79 at 6 hours) because the daily rhythm starts to matter. This is exactly the opening XGBoost uses: it blends all three and learns how to weigh them for each horizon.

8. The physics model: how attention decays after a spike

The decay model treats a spike like an impulse that relaxes back to a baseline, fits one decay curve to hundreds of real spikes, and uses it to predict how fast a spiking page will cool. It explains *why* forecasting helps; on its own it is slightly worse than the standard rule, but it feeds XGBoost as an input.

Why we built it

- **It answers the question `last_hour` gets wrong.** `last_hour` assumes a page at 20,000 views will stay at 20,000. After a spike that is wrong within the hour.
- **It is interpretable.** One number (a half-life) summarises how fast public attention moves on. Judges can check it against their own experience of news.
- **It brings the physics teammate's method in.** Impulse and relaxation is standard physics (radioactive decay, a capacitor discharging, a struck bell). Applying it to attention is the "time" theme in its purest form.
- **It feeds the ML model.** Its prediction is a feature in XGBoost (`decay_pred`), so the physics shapes the final model even where it does not win alone.

The model

For a page at hour `now`, looking at its last 24 hours:

- **baseline** = median of those 24 hours (its normal level; the median ignores the few spike hours)
- **excess** = views now − baseline (never below 0)
- **t** = hours since the page's busiest hour in that window
- **prediction at now + h** = baseline + excess × f(t + h) / f(t)

f is the decay curve. The ratio f(t + h) / f(t) says what fraction of today's excess survives h more hours. Two shapes are tried:

Shape	Formula	Meaning
Exponential	f(t) = exp(−t / τ)	A constant fraction of the excess leaves every hour, however old the spike is. Half-life = τ ln 2.
Power law	f(t) = (1 + t / τ)^−α	Decay slows as the spike ages: early readers leave fast, a long tail keeps arriving through links and search. Half-life (from the peak) = τ (2^(1/α) − 1).

One useful consequence: for the exponential, f(t + h) / f(t) = exp(−h / τ) does not depend on t. The page's age does not matter, only the horizon. For the power law the same h removes less of the excess the older the spike is. Studies of online attention (for example YouTube views, Crane & Sornette 2008) often find power-law relaxation, so letting the data choose between the two was the honest option.

Worked example (illustrative page, our fitted curve: power law, α = 1.79, τ = 0.87 h). Baseline 500 views/h; peak 20,000 two hours ago; now 4,000. Excess = 3,500. One hour ahead: factor = f(3) / f(2) = ((1 + 3/0.87) / (1 + 2/0.87))^−1.79 ≈ 0.59, so the prediction is 500 + 3,500 × 0.59 ≈ 2,550. `last_hour` would say 4,000. Compare the first hour after the peak: f(1) / f(0) ≈ 0.25, so three quarters of the excess goes. Power-law decay slows as the spike ages.

Only for pages that actually spiked

The first version applied the formula to every page. For steady pages, "baseline = 24 h median" ignores the daily rhythm (a page always busier in the evening), so the model lost badly to `last_hour` on small caches: −97% of the gap at N = 1,000 in an early run. Now the decay is applied only when the page's 24 h peak is **≥ 3× its baseline and at least 100 views above it** (`SPIKE_FACTOR` , `SPIKE_MIN_EXCESS`). Every other page is predicted as its latest views, exactly like `last_hour` . Gating cut the damage from −97% to −34% at 1,000 pages and from −63% to −16% at 10,000, but did not remove it (see "How it scores on its own" below).

How the curve is fitted (`fit_decay.py`)

1. **Find spikes in the training hours only** (before the test period), with the same rule as the backtest.
2. **Line each spike up at its peak.** The peak is the busiest hour within 6 hours of onset (some spikes build for an hour or two).

3. **Normalise.** Subtract the page's baseline (median of the 24 hours before onset) and divide by the excess at the peak. Every spike now starts at 1 and falls towards 0, whether it peaked at 2,000 or 200,000.
4. **Take the median across spikes** at each hour since the peak (only hours with ≥ 5 spikes). The median resists odd pages, such as one with a second news story the next morning.
5. **Fit only the decay phase**: from the peak until the median stops falling (checked after hour 3), at most 12 hours.
6. **Fit both shapes in log space** with `scipy.optimize.curve_fit` and keep the one with lower log error. Log space gives hour 5 (value ~0.02) as much say as hour 1 (value ~0.4); in plain space the first two hours would decide everything.
7. **Measure the daily echo separately**: the highest median 18–30 hours after the peak, compared with the lowest point before it.
8. Save `models/decay_params.json` and the chart `out/decay_fit.png`.

What we found (1,429 spikes from the training days)

- **Shape: power law, not exponential.** The best fit is f(t) = (1 + t / 0.87)^−1.79, fitted on the first 10 hours after the peak. The exponential (τ = 1.84 h) is too slow in the first hours and too fast later; on the log-scale chart it is a straight line that misses the curve's bend.
- **Half-life: about 25 minutes.** The median spike keeps about 30% of its excess after 1 hour, 12% after 2, 7% after 3 and 4% after 4. Public attention moves on very fast.
- **A floor, then an echo.** From about 8 to 18 hours the excess stops falling, at around 1.3% of the peak. Then it rises to **2.0% of the peak 23 hours after it**, 1.5× its low point, and falls away again after 25 hours. The likely reason is people returning at the same time the next day (routines, time zones, follow-up coverage); we measured the echo but did not test why it happens.
- **What the power law means.** Exponential decay is memoryless: the same share leaves every hour. A power law says the decay slows as the story ages: the casual crowd leaves first, then a smaller, steadier audience keeps arriving via links and search. This matches what studies of online attention often report.

How it scores on its own

The gated decay model is still slightly but significantly **worse** than `last_hour`: −0.19 points at 1 hour (interval −0.22 to −0.15, better in 0% of hours) and −0.11 at 6 hours. It recovers +8% of the gap only at 100,000 pages. It also serves less spike traffic (91.8% vs 93.5%). Two likely reasons, both visible in the data:

1. **It cuts spikes that are still rising.** At the onset hour the page's busiest hour is "now", so t = 0 and the model predicts a 75% drop. Tom Skerritt went from about 54,000 to 97,628 views in the next hour; the decay model would have demoted him exactly then.
2. **It uses the median spike's curve for every spike.** The pages that matter most for the cache are the biggest stories, which may hold attention longer than the median spike.

This is fine for the story: the physics explains *how* attention fades, and XGBoost decides *when* to trust it (as the `decay_pred` feature), which is where the decay curve earns its keep.

A bug worth telling as a story

The first fit used all 30 hours after the peak in one curve. The next-day rebound pulled the fit into nonsense: τ = 0.02 h, a half-life of 6 minutes. A smooth decay cannot also rise again at 24 h. The fix was to fit the decay phase alone and describe the echo as its own effect. This is a good example of checking a model against a plot rather than trusting the optimiser.

Limitations

- **It cannot see a spike coming.** Before the peak, the busiest hour is "now", so t = 0 and the model predicts immediate decay even if the page is still rising. It helps on the way down, not on the way up.
- **One curve for every page.** A celebrity death, a football result and a viral meme probably decay differently. We fit one median curve; a per-category curve is future work.
- **The echo is measured, not used.** The prediction formula has no 24 h rebound term. (XGBoost can still learn it from `lag_23` and `lag_24`.)
- **Baseline from the median of 24 hours.** For a page that is spiking for most of the window, the median is inflated and the excess understated.

- **One week of news.** The curve comes from the spikes in the training days only (`fit_decay.py` prints how many); a different week could give a somewhat different half-life.

9. The machine-learning model: XGBoost

XGBoost learns, from the training days, how each page's last 25 hours predict its views 1, 2, 3 or 6 hours later. It combines the persistence of `last_hour`, the averaging of `last_24h_mean`, the daily rhythm and the physics decay model, and learns when to trust each.

Why XGBoost (and not a transformer)

Need	Why gradient-boosted trees fit
Tabular features, millions of rows	Trees train in minutes on a laptop CPU
Very skewed numbers (1 to 100,000+ views)	Trees split on thresholds, so scale barely matters; we also use log views
Missing hours	XGBoost learns a default branch for missing values (NaN)
Interactions ("falling fast **and** peaked 2 h ago")	Each tree path is an interaction
Explainability	Feature importances are printed and saved to `models/xgb_report.json`

We considered a transformer over each page's time series. With about one week of hourly data it would have little to learn from, would take hours to train, and would be harder to explain. The honest sentence for judges: *"With a week of data, a well-featured tree model is the right tool; a sequence model is the natural next step with months of data."*

What it predicts

One model per horizon (`models/xgb_h1.json`, `h2`, `h3`, `h6`). Each predicts **log(1 + views at now + h)** for a page. Log because views span five orders of magnitude: in plain units, the error on the top page would swamp everything else. The backtest converts back with exp(x) − 1 and caches the top N. If a horizon has no model (too little data), the nearest trained horizon is used.

Features (`predictors/xgb_features.py`)

All features come from a **window**: a matrix with one row per page and one column per hour, for the 25 hours `now − 24 … now`. A page absent in an hour we have = 0; an hour we do not have = NaN. Views are used as log(1 + views).

Feature	What it is	Why it helps
`lag_0` … `lag_6`	Log views now and in each of the 6 hours before	Recent level and short-term trend; `lag_0` is `last_hour`
`lag_12` , `lag_18`	12 and 18 hours ago	`lag_18` is the same clock hour as the target for 6 h ahead, yesterday
`lag_23` , `lag_24`	23 and 24 hours ago	Same clock hour as the 1 h ahead target, yesterday; daily rhythm and the spike echo
`diff_1` , `diff_3` , `diff_24`	Change in log views over 1, 3, 24 hours	Rising or falling, and how fast
`mean_24` , `max_24` , `std_24`	Mean, max, spread over the previous 24 h	Normal level, recent peak, how volatile the page is
`hours_since_peak`	Hours since the busiest hour in the window	Where the page is in its decay
`peak_over_now`	log(peak) − log(now)	How far it has already fallen
`hours_present`	Hours with any views	Long-lived vs brand-new page
`decay_pred`	The physics model's 1-hour-ahead forecast (spike-gated)	Hands the trees the decay curve directly
`is_spike`	1 if the page passes the decay model's spike test	Lets the trees treat spiking pages differently
`hour_of_day` , `day_of_week`	Of `now` , in UTC	Global daily pattern (weekly barely learnable with one week)

The same file is used by training and by prediction, so they always compute identical features. A mismatch between the two is one of the most common silent bugs in ML systems.

Which pages get scored

Scoring every page that ever appeared would be slow and pointless. Each hour, pages are ranked by **max(latest views, 24 h mean)** and the top 200,000 are scored (`select_candidates`). Using the max means a page that just jumped and a page that is steadily big both get in.

How it is trained (`train_xgb.py`)

1. Keep only hours **before the test period**; training targets also stay before it (purging, section 2).
2. The **universe** is every page that was in the top 60,000 of at least one training hour. Build a pages × hours matrix for them.
3. Use every 2nd training hour as a `now` (`--stride 2`); neighbouring hours are nearly duplicates, so this halves the time and loses little.
4. Skip a `now` if that hour is missing, or if less than 75% of its 25-hour window exists (a download gap would make the features nonsense). For each horizon also skip it if the target hour is missing.
5. For each `now`, take the **top 30,000 candidates plus 10,000 random other pages**. The random ones teach the model about small pages that might suddenly rise; otherwise it would only ever see pages that are already big.
6. **Weight** each row by 1 + log(1 + current views). Hit rate counts views, so getting big pages right matters more, but the log keeps a few giant pages from dominating.
7. Hold out the **last 20% of training hours, in time order**, for validation. Training stops when validation error has not improved for 40 rounds (early stopping, up to 600 trees). A random split would leak: rows from the same hour would sit on both sides.
8. Print validation error against "same as now" (`lag_0`) and the top features; save the models and `xgb_report.json`.

Settings: depth 8, learning rate 0.05, 80% row and column subsampling, minimum child weight 10, histogram tree method. These are sensible defaults; we did not tune them (no time, and tuning on 72 test hours would overfit).

Limitations

- **Little training data.** At most a few days of training hours: everything before the 72-hour test period, minus the first 24 hours (they only fill the feature window) and any gaps. `train_xgb.py` prints how many hours it used and how much it beat "same as

now" on validation; quote those if asked.

- **One week cannot teach weekly patterns** (weekday vs weekend), and `day_of_week` takes only a few values.
- **It does not help on spikes.** XGBoost serves 92.8% of spike traffic vs 93.5% for `last_hour`. Its gain comes from ordinary pages. A likely cause: like the decay model, it sometimes demotes a spike that is still rising.
- **Train/serve difference.** Training rows come from the top-60k universe; at test time candidates come from all pages in the history. Pages that were never in a training hour's top 60k are slightly out of distribution.
- **`decay_pred` is always the 1-hour-ahead decay forecast**, even for the 6-hour model. The trees can rescale it, but a horizon-matched feature would be cleaner.
- **Untuned hyperparameters.** Probably some gain left, but tuning needs a separate validation period we do not have.
- **It predicts the spike's fade, not its arrival.** It cannot know about tomorrow's news before anyone reads it.

10. Charts and key numbers (`make_charts.py`)

`make_charts.py` turns the backtest output into seven slide-ready charts and one text file of every number the pitch needs. Rerun it after every backtest; never copy a number onto a slide from memory.

It reads `out/results.csv`, `out/hourly.csv`, `out/demo.json` and `models/decay_params.json`, and writes to `out/charts/`. Colours are fixed across all charts: XGBoost blue, decay orange, baselines grey, the oracle ceiling dark. Keep the same colours on any slide you make by hand.

Which chart goes where

Chart	Shows	Slide	What to say
`1_vs_standard_rule.png`	Each method's difference from `last_hour` on the same hours, at 1 h and 6 h, with 95% block-bootstrap intervals; "n.s." where not significant	**Main result**	"Same hours, paired: XGBoost +0.12 at 1 hour, +1.28 at 6 hours, both clear of zero. Note the 24-hour average also beats the standard rule at 6 hours."
`3_foresight.png`	Advantage over `last_hour` as the horizon grows from 1 to 6 hours, for XGBoost, the decay model and the oracle	**The time story**	"The further ahead you must decide, the more prediction is worth: +0.12, +0.24, +0.50, +1.27 points."
`5_decay_fit.png`	Median of 1,429 spikes on a log scale, fitted decay phase shaded, both fits, daily echo marked	**The physics**	"Attention halves every 25 minutes, follows a power law, then echoes 23 hours later."
`2_gap_recovered.png`	Share of the gap to the oracle each model recovers, per cache size	**Where it helps**	"XGBoost: level at 1,000 pages, +10% at 10,000, +30% at 100,000. The decay model alone only helps at 100,000."
`4_spikes.png`	Share of spike pages' views in their first 6 h served from cache	**Honesty slide**	"Spikes are not where the gain is: the standard rule already serves 93.5% of spike traffic. Our gain comes from everything else."

Chart	Shows	Slide	What to say
`6_hit_rate_absolute.png`	Absolute hit rates with per-method intervals, zoomed	Appendix	Use only if asked "what are the actual hit rates?"
`7_spike_example.png`	Tom Skerritt on 22 Oct: almost nothing at 19:00, 97,628 views at 21:00; both caches had him from the first spike hour	Hook or appendix	A concrete spike makes the problem real; it also shows why spikes are easy at 10,000 pages

Reading chart 1 correctly

Each dot is the average per-hour difference from `last_hour`, in percentage points. The line through it is the 95% interval. If the line crosses zero, the chart writes "n.s.". The oracle's dot shows the ceiling in the same units, so the audience can see how much of the possible gain we took. Do not compare the absolute bars in chart 6 to argue significance (section 6).

`key_numbers.md`

A plain-text list, in sentences, of: hit rates at N = 10,000; the gap and the share recovered for each model and cache size; the paired differences with intervals and "significant / NOT significant"; the share of hours each model was better; the foresight table; spike hit rates; the decay half-life in minutes and the echo. Paste from it; it is regenerated every run, so numbers on slides always match the code.

Design notes

- Every chart title states the finding, not the topic ("After a spike, attention halves every N minutes" rather than "Decay fit").
- Values are written next to dots, never on top of reference lines.
- Hit-rate axes are zoomed because the differences are small. Say so out loud if anyone could read the zoom as exaggeration; chart 2 and chart 1 express the same result on honest scales (share of gap, paired difference).

11. Results, and what they mean

XGBoost beats the standard rule on the same 72 unseen hours: by a small but consistent +0.12 points at 1 hour ahead, and by +1.28 points at 6 hours ahead, better in every single hour. The gain comes from ordinary pages, not spikes. All numbers are from `key_numbers.md` and the charts.

A sanity check worth knowing: the baselines and the oracle score exactly the same as in the earlier 123-hour run (e.g. `last_hour` 40.47%). That confirms the test hours did not change; only the models did.

Hit rate, room for 10,000 pages, 1 hour ahead

Method	Hit rate (95% interval)	Paired difference from `last_hour`
Oracle (ceiling)	41.66% (41.32–42.05)	+1.19 (+1.10 to +1.28)
XGBoost + decay feature	40.59% (40.29–40.92)	**+0.12 (+0.10 to +0.14)**, better in 94% of hours
`last_hour`	40.47% (40.17–40.81)	—
Decay model (gated)	40.28% (39.99–40.61)	−0.19 (−0.22 to −0.15), better in 0% of hours
`last_24h_mean`	39.12% (38.89–39.38)	−1.35
`same_hour_yesterday`	36.66% (36.43–36.93)	−3.81

Note how the per-method intervals for XGBoost and `last_hour` overlap almost completely, yet the paired interval (+0.10 to +0.14) is far from zero. This is section 6 in action.

6 hours ahead, room for 10,000 pages

Method	Paired difference from `last_hour`
Oracle (ceiling)	+4.20 (+3.60 to +4.61)
XGBoost + decay feature	**+1.28 (+0.93 to +1.49)**, better in 100% of hours
`last_24h_mean`	+0.53 (from chart 1)
Decay model (gated)	−0.11 (−0.16 to −0.06), better in 19% of hours
`same_hour_yesterday`	−0.79 (from chart 1)

Share of the gap recovered, 1 hour ahead

Cache size	Decay model (gated)	XGBoost
1,000	−34%	+0%
10,000	−16%	+10%
100,000	+8%	+30%

Advantage over `last_hour` by horizon, 10,000 pages

Hours ahead	XGBoost	Decay model	Oracle	XGBoost's share of the gap
1	+0.12	−0.19	+1.19	10%
2	+0.24	−0.26	+1.95	12%
3	+0.50	−0.20	+2.66	19%
6	+1.27	−0.12	+4.19	30%

(The 6-hour figure is +1.27 here and +1.28 in the paired test, and the oracle +4.19 vs +4.20, only because `results.csv` rounds hit rates to four decimals. They are the same number.)

Spikes, room for 10,000 pages, 1 hour ahead

Method	Share of spike traffic served (first 6 h)	Median hours late
Oracle	99.6%	0
`last_hour`	**93.5%**	0
XGBoost	92.8%	0
Decay model	91.8%	0
`last_24h_mean`	84.6%	1
`same_hour_yesterday`	47.3%	14

Six findings and why they happen

1. **The standard rule is already good.** At 1 hour the most anyone could add is 1.19 points. Hourly attention is persistent for the vast majority of pages.
2. **Prediction is worth more the further ahead you decide.** XGBoost's advantage grows from +0.12 to +1.27 points, and its share of the gap from 10% to 30%. Over 6 hours, `last_hour` goes stale: even the plain 24-hour average beats it then (+0.53). XGBoost blends recent level, longer averages and the daily rhythm, and learns a different blend for each horizon. **This is the headline for the time theme.**
3. **Bigger caches benefit more.** Level at 1,000 pages, +10% of the gap at 10,000, +30% at 100,000. The top 1,000 are huge, steady pages any rule gets right. Further down, a single hour's count is noisier, and a model that knows each page's normal level ranks them better. (Our explanation of the pattern, not something we tested separately.)
4. **Spikes are not where the gain is.** At 10,000 pages, every method except the two slow baselines caches a spike within its first hour (median lag 0) and serves over 90% of spike traffic. `last_hour` is slightly best (93.5%). The 6.1 points it misses against the oracle are probably mostly onset hours, which nobody can foresee from page views alone. This changes our pitch from "we catch spikes" to "we rank everything else better, and spikes teach us how attention fades".
5. **Physics alone loses, physics inside the model wins.** The gated decay model is significantly worse than `last_hour` on its own (−0.19), mostly because it demotes spikes that are still rising (section 8). Inside XGBoost, the same curve is one input among many, and the trees learn when to trust it.

6. **The decay curve is a clean physical result.** 1,429 spikes, a power law with a 25-minute half-life, a floor at about 1% of the peak, and a daily echo at 23 hours.

Still worth checking before the pitch

- XGBoost's top features in `models/xgb_report.json`: a good slide line is "the model leans most on …". If `decay_pred` is near the top, that is the bridge between the physics and the ML.
- Paired results at 100,000 pages (`make_charts.py` computes them for N = 10,000 by default); if the +30% is significant there, it is the strongest single claim.
- One concrete page that XGBoost cached and `last_hour` did not, from `demo.json`, as an example for the main-result slide.

12. Limitations and future work

The method is sound, but the evidence is one week of data and a simplified model of a cache. Saying this before a judge does is a strength: it shows we know exactly what the numbers can and cannot support.

Limitations, ranked by how much they matter

#	Limitation	Effect on our claims	What would fix it
1	**One week of data, three test days**	The result could differ in another week (a quieter or busier news cycle)	Months of history; test on several separate weeks
2	**Hourly decisions, not request-level caching**	We show *which pages deserve the cache this hour*, not a drop-in replacement for a CDN's eviction policy	Simulate request streams within each hour; combine prediction with LRU (e.g. use the forecast for pre-warming)
3	**Hit rate within the top 150k pages**	Absolute hit rates are higher than real-world ones; method differences barely change	Include the full tail (costly)

#	Limitation	Effect on our claims	What would fix it
4	**Page views, not bytes or requests**	A real CDN also caches images, scripts and API calls; page sizes differ	Weight by bytes; add media files
5	**Cannot predict a spike's onset**	Spike traffic is already 93.5% served by the standard rule; what's left is mostly onsets, and neither of our models closes it	External signals: trending searches, news feeds, social media, links from the Main Page
6	**Thin training data for XGBoost**	Model is probably under-trained; gains may be underestimated	More days; walk-forward retraining every day
7	**Untuned settings** (XGBoost hyperparameters, spike thresholds, 6 h block length)	Some numbers could move a little	A separate validation week for tuning
8	**One decay curve for every page**	Misses differences between kinds of news	Fit curves per topic or per spike size
9	**English Wikipedia only**	Other languages and sites may behave differently	Rerun on `de`, `fr`, `ja`… (same code, change the domain filter)
10	**Download gaps**	A few hours missing; handled as unknown, but they reduce training and test hours	Complete download (the full set covers 16–24 Oct)

Assumptions we made on purpose

- **The cache is refilled once per hour, all at once.** Simple and comparable across methods. Real caches update continuously.
- **Missing page in an hour we have = 0 views.** True for pages below the top-150k cut (they had few views), so the error is small.
- **A spike is ≥ 1,000 views and ≥ 10× the 24 h median.** Strict enough to mean "something happened", loose enough to catch many events every day (the backtest prints the count). Written into `INTERFACES.md` before any results and used everywhere.

- **Same test period for every method, no retraining during it.** Retraining every day would likely help XGBoost; we chose the simpler, more conservative setup.

Future work, if we had another week

1. **More data and rolling evaluation.** Download a month; retrain each day on the past and test on the next day; report results for every day, not one block of three.
2. **Predict onsets with outside signals.** Link Wikipedia to news headlines or Google Trends; a page linked from the Main Page's "In the news" box is a strong hint.
3. **Hybrid cache policy.** Keep LRU for request-level eviction and use our forecast to pre-warm and to evict fading spikes early. Measure on simulated request streams.
4. **Sequence models.** With months of data, a transformer or temporal convolution over each page's history (the original idea) becomes worth trying, with XGBoost as the bar to beat.
5. **Decision-aware training.** Train the model to rank pages near the cut-off correctly (a ranking loss), rather than to predict views; that targets the hit rate directly.
6. **Cost model.** Turn hit rate into money or latency: each extra hit saves a trip to the origin servers. That turns "+1.2 points" into a number an operator would act on.

13. Questions judges are likely to ask

HRT judges think like quant researchers: they will probe leakage, significance, baselines and whether the gain is worth anything. Each answer below is short enough to say out loud and points to the section with the detail.

On validity

"How do you know there's no lookahead?" Every method gets only rows with hour ⩽ now. Models are fitted only on hours before the test period, and training targets also stop before it. All three scripts share one `--test-days` setting. (Sections 2, 5.)

"Is the improvement significant?" Yes. We compare methods on the same hours, using the per-hour difference from the standard rule and a 6-hour block bootstrap because neighbouring hours are correlated. At 6 hours ahead XGBoost is +1.28 points, interval +0.93 to +1.49, better in every test hour. At 1 hour it is +0.12, interval +0.10 to +0.14, better in 94% of hours. (Section 6.)

"Your confidence intervals overlap." Those are per-method intervals; they include the hour-to-hour swing every method shares. The paired test removes it. Overlapping separate intervals do not mean no difference.

"Did you tune on the test set?" No. Hyperparameters are defaults, the spike rule was fixed in `INTERFACES.md` beforehand, and XGBoost's early stopping uses the last 20% of the training hours.

"Three days is not much." Agreed; it is our main limitation. The pattern (gain grows with horizon, strongest for big caches) holds across every cell, which is harder to get by luck than one number. Months of data and rolling evaluation is the first next step. (Section 12.)

On the baselines

"Is `last_hour` a fair baseline?" It is the hourly version of LRU, the default in real caches, and it is the strongest of our three simple rules. We beat the best baseline, not a straw man.

"What's the oracle?" Perfect foresight: it caches what actually turned out to be the top N. Nothing can beat it, so it tells us how much was possible. Like Belady's optimal algorithm in caching theory.

"1.3 points sounds tiny." The total possible gain at 6 hours is 4.20 points; we get 30% of it, and at 100,000 pages we get 30% even at 1 hour. In absolute terms, 1 point of hit rate is 1% of every view in our data. `demo.json` stores `total_views` for each replay hour, so before the pitch compute 1.28% × a typical hour's total × 24 and quote that as "page views per day served from cache instead of the origin servers".

On the models

"Why XGBoost and not deep learning?" One week of hourly data, millions of tabular rows, missing values, and a need to explain the model. Trees fit all of that. With months of data a sequence model becomes worth trying. (Section 9.)

"Your physics model is worse than the baseline. Why keep it?" Because it measures something real: across 1,429 spikes, attention decays as a power law with a 25-minute half-life and echoes 23 hours later. As a cache rule on its own it is too blunt; it demotes spikes that are still rising (−0.19 points). As an input to XGBoost it lets the trees decide when the decay applies. Physics for the mechanism, ML for the decision. (Section 8.)

"Why does the gain grow with the horizon?" The further ahead, the staler the last hour: spikes fade and the daily rhythm moves attention. Those are the patterns the model knows. At 1 hour, the last hour is still nearly right.

"Can you predict a spike before it happens?" No, and we don't claim to. We predict how a spike fades and how the daily rhythm moves attention. Predicting onsets needs outside signals such as news feeds. (Section 12.)

"So does your model help with spikes?" No, and we found that out by measuring it. At 10,000 pages the standard rule already serves 93.5% of spike traffic; XGBoost serves 92.8%. Big spikes enter the cache within their first hour under any reactive rule. Our gain is on ordinary pages, especially further ahead and for larger caches. (Section 11, finding 4.)

"Why weight training rows by page size?" Hit rate counts views, so mistakes on big pages cost more. The weight is logarithmic so a few giant pages don't dominate.

On usefulness

"Would Wikipedia use this?" Not as a replacement for request-level caching. As a planner it could pre-warm edge caches before the evening peak and drop fading spikes early. The same idea applies to any system that must commit capacity ahead of demand: CDNs, streaming, ad servers, compute scheduling.

"What decision does this optimise?" Which N items to hold given limited capacity, decided h hours before demand arrives. We score the decision (views served), not just forecast error.

If you don't know

Say what you would check and how: "We haven't measured that; we'd rerun the backtest with X, which takes about Y minutes." A precise plan is better than a guess.

14. Pitch skeleton and glossary

The pitch should follow the evidence in the order a sceptical judge needs it: problem, fair test, result, mechanism, honesty. Fill every number from `key_numbers.md` after the full rerun.

Slide skeleton (about 5 minutes)

#	Slide	Content	Chart	Suggested speaker
1	Hook	One real spike: Tom Skerritt goes from almost nothing to 97,628 views an hour on 22 Oct. Every hour, a cache must guess which pages will matter next.	`7_spike_example.png`	Design Eng
2	The decision	Limited cache, millions of pages; choose the top N, h hours ahead. Standard rule: keep what was popular last hour.	(simple diagram)	Design Eng
3	The fair test	Real Wikipedia data, 8.5 days; train on the past, replay 3 unseen days; score views served; oracle as the ceiling.	pipeline diagram (section 4)	Computing
4	Main result	Same hours, paired: XGBoost +0.12 at 1 h (wins 94% of hours), +1.28 at 6 h (wins every hour), intervals clear of zero.	`1_vs_standard_rule.png`	Computing
5	Time is the story	Advantage grows with horizon: the earlier you must commit, the more prediction is worth.	`3_foresight.png`	Computing

#	Slide	Content	Chart	Suggested speaker
6	The physics	1,429 spikes: attention halves in about 25 minutes (power law), then echoes 23 hours later. Alone it loses; as an input to XGBoost it helps.	`5_decay_fit.png`	Physics
7	Where it helps	Biggest gains for large caches: 30% of the possible gain at 100,000 pages. Not from spikes: the standard rule already serves 93.5% of spike traffic.	`2_gap_recovered.png`, `4_spikes.png`	EIE
8	Honest limits and next steps	One week of data; can't foresee onsets; next: months of data, news signals, pre-warming policy.	none	EIE
9	Close	One sentence: "We turned a forecast into a better decision, and measured exactly how much better."	none	anyone

Keep a live demo (the replay day from `demo.json`) as a backup or for the Q&A if the Design Eng page is ready.

One-sentence versions

- **What it is:** We predict Wikipedia attention hours ahead and use it to decide what a limited cache holds.
- **What we found:** Prediction beats the standard caching rule on every test hour at 6 hours ahead, and the further ahead you must decide, the bigger the win.
- **Why it works:** The last hour goes stale; attention fades after spikes as a power law and follows a daily rhythm. The model blends these and learns a different blend for each horizon.
- **What surprised us:** Spikes are not where the gain is; reactive caching already handles them. The gain is in ranking everything else.
- **Why you can trust it:** Real data, strict past-only training, same-hours paired testing, and a perfect-foresight ceiling.

Glossary

Term	Meaning
Backtest	Replaying past hours to score what a method would have done, using only information available at the time
Baseline	A simple rule to beat; ours are `last_hour`, `last_24h_mean`, `same_hour_yesterday`
Block bootstrap	Resampling runs of consecutive hours (6 here) to get confidence intervals that respect correlation in time
Cache	Fast storage holding a limited number of pages (N)
Catch lag	Hours from a spike's onset until a method caches the page (kept, but too coarse at N = 10k)
Daily echo	The rise in a spiked page's attention about 24 hours after its peak
Decay phase	The first hours after a spike's peak, where attention falls steadily; the part we fit
Early stopping	Stop adding trees when validation error stops improving (40 rounds here)

Term	Meaning
Excess	Views above a page's normal (baseline) level
Feature	An input number the model sees, e.g. views 3 hours ago
Gap	Oracle hit rate minus `last_hour` hit rate: the most any method could add
Half-life	Time for a spike's excess attention to halve
Hit rate	Share of views served by cached pages
Horizon (h)	How many hours ahead the cache is decided
Leakage	Future information sneaking into training or prediction
LRU / LFU	Least recently / least frequently used: the classic cache rules our baselines stand in for
Oracle	Perfect foresight; the ceiling
Paired test	Comparing two methods hour by hour on the same hours, so shared swings cancel
Purging	Dropping training rows whose targets fall in the test period
Share of gap recovered	(model − last_hour) ÷ (oracle − last_hour)
Spike	≥ 1,000 views and ≥ 10× the previous 24 h median, counted at its first hour
Spike gating	Applying the decay formula only to pages whose 24 h peak is ≥ 3× their baseline and ≥ 100 views above it
Spike hit rate	Share of spike pages' views in their first 6 hours that came from cache
Walk-forward	Evaluation that moves forward through time, never using the future
XGBoost	Gradient-boosted decision trees: many small trees, each correcting the previous ones' errors